

Aoudia Sofia
Delecheneau Camille
Philippot Lucie

Rapport de projet

[Phase 1]

Avant de débiter la création de notre projet, nous avons longuement discuté du design de notre restaurant, de ce que nous allons vendre, du thème et de la charte graphique. Nous avons plusieurs fois changé d'idée avant de tomber d'accord sur notre restaurant KØLD, axé sur le thème du froid.

Sofia a créé l'architecture du projet en préparant tous les fichiers dont nous avons besoin. Nous avons lancé notre projet, Camille s'est d'abord occupé de la page de connexion. Pendant ce temps, Sofia s'occupait de la page d'accueil et Lucie de la page d'inscription. Puis, nous avons un peu tous travaillé sur différentes pages, nous pouvions faire une partie du code, puis les autres complétaient.

Au début de notre projet, nous avons rencontré plusieurs problèmes. Tout d'abord, l'environnement de travail de Lucie rencontrait des problèmes de connexion entre github et vs code (des erreurs de push et pull). Pour ne pas bloquer l'avancée du projet, nous avons temporairement partagé les fichiers par mail avant de réussir à régler ce problème. Nous avons également rencontré un manque de coordination et d'organisation, qui nous a amenés plusieurs fois à modifier le même fichier en même temps ou même à coder la même chose. Nous nous sommes ensuite mieux organisés, nous avons mieux communiqué et mieux réparti les tâches, et nous n'avons plus rencontré ce problème.

En conclusion, pour l'instant, nous aimons travailler sur ce projet qui rend ce que nous codons très visuel. Il nous a permis de faire travailler notre imagination et notre sens de l'organisation de groupe.

[Phase 2]

Pour la deuxième phase de notre projet, nous nous sommes concentrés sur la gestion des données de notre restaurant. Pour stocker nos informations, nous avons choisi d'utiliser des fichiers au format JSON. C'est un format de données qui a l'avantage d'être très simple à lire pour nous, et facile à manipuler dans notre code.

Afin de garder un projet bien organisé, nous avons réparti nos données dans trois fichiers distincts.

Tout d'abord, nous avons créé le fichier **users.json** qui gère tous les comptes. Chaque personne y possède un identifiant unique (id), un nom, un prénom, et des

informations de contact (email, téléphone, adresse). Surtout, nous avons ajouté un champ "rôle" qui nous permet de différencier les utilisateurs normaux des administrateurs, des restaurateurs et des livreurs.

Ensuite, nous avons construit notre carte dans le fichier **produits.json**. Ce fichier est séparé en deux grandes listes: les "plats" et les "menus". Les plats regroupent tout ce que nous vendons à l'unité (les bowls, les wraps, les salades, les boissons et les desserts). Chaque plat a son nom, sa description, son prix et le lien vers son image. Pour les "menus", nous voulions éviter de recopier les mêmes informations plusieurs fois. Donc, un menu possède un prix global et une liste (plats_inclus) qui contient simplement les identifiants des plats qui le composent.

Enfin, le fichier **commandes.json** s'occupe de l'historique et du suivi des commandes. C'est le fichier central qui relie les utilisateurs aux produits. Une commande enregistre l'identifiant du client (user_id), la liste exacte des articles choisis (en précisant s'il s'agit d'un plat ou d'un menu, son identifiant et la quantité), l'adresse de livraison et l'heure. Nous avons aussi intégré des statuts pour suivre l'évolution de la commande (statut_commande en attente, en livraison, terminée...) et du paiement. Lorsqu'une commande part en livraison, l'identifiant du livreur qui s'en charge est directement rattaché à la commande.

Cette organisation nous a demandé de bien réfléchir en amont pour relier logiquement nos fichiers entre eux (par exemple, relier un identifiant de commande à un identifiant de livreur et d'utilisateur), mais elle correspond exactement à ce qu'il nous fallait pour faire fonctionner la prise de commande de KØLD de manière fluide.

Voici le détail complet de la structure de données pour chaque élément, tel qu'il est défini dans nos fichiers JSON :

Structure d'un Utilisateur (user)

Un utilisateur est défini par les attributs suivants:

- **id**: Identifiant unique du compte (nombre).
- **nom**: Nom de famille (chaîne de caractères).
- **prenom**: Prénom (chaîne de caractères).
- **email**: Adresse email (chaîne de caractères unique).
- **password**: Mot de passe (chaîne de caractères).
- **tel**: Numéro de téléphone (chaîne de caractères).
- **role**: Le rôle attribué, qui peut être "admin", "restaurateur", "livreur" ou "client" (chaîne de caractères).
- **adresse**: Adresse postale de l'utilisateur (chaîne de caractères).
- **infosup**: Informations supplémentaires sur l'adresse (chaîne de caractères).
- **statut**: Le statut de l'utilisateur, "Moldu", "Premium" ou "VIP" (chaîne de caractères).

- **remise:** Montant de remise applicable (chaîne de caractères).
- **bloqué:** Indique si le compte est restreint ou non (booléen: true ou false).

Structure d'un Plat (plat)

Un plat à l'unité est défini par les attributs suivants:

- **id:** Identifiant unique du plat (nombre).
- **nom:** Le nom du produit (chaîne de caractères).
- **description:** Les ingrédients ou la description du plat (chaîne de caractères).
- **prix:** Le prix de vente (nombre décimal).
- **categorie:** La famille du produit, par exemple "bowls", "wraps", "salades", "desserts" ou "boissons" (chaîne de caractères).
- **image:** Le chemin vers le fichier image correspondant, par exemple "../images/poke_saumon.png" (chaîne de caractères).

Structure d'un Menu (menu)

Un menu regroupant plusieurs articles est défini par les attributs suivants:

- **id:** Identifiant unique du menu (nombre).
- **nom:** Le nom du menu (chaîne de caractères).
- **description:** Une courte phrase décrivant ce que le menu contient (chaîne de caractères).
- **prix:** Le prix global du menu (nombre décimal).
- **plats_inclus:** Une liste contenant les identifiants (id) des plats qui composent ce menu (tableau de nombres).

Structure d'une Commande (commande)

Une commande relie les utilisateurs et les produits, et est définie par les attributs suivants:

- **id:** Identifiant unique de la commande (nombre).
- **user_id:** L'identifiant du client qui a passé la commande (nombre, ou null si non renseigné).
- **articles:** La liste des produits commandés (tableau d'objets). Chaque article précise :
- **type:** S'il s'agit d'un "plats" ou d'un "menu" (chaîne de caractères).
- **id_produit:** L'identifiant du produit ou du menu commandé (nombre).
- **quantite:** Le nombre d'unités pour ce produit (nombre).
- **quand:** L'échéance de la commande, comme "maintenant" ou "plus tard" (chaîne de caractères).
- **adresse_livraison:** L'adresse où la commande doit être livrée (chaîne de caractères).
- **statut_paiement:** L'état du règlement, par exemple "payé", "en attente" ou "remboursé" (chaîne de caractères).

- **statut_commande:** L'état d'avancement de la commande, par exemple "terminée", "en livraison", "a livrée", "en attente" ou "abandonnée" (chaîne de caractères).
- **date_heure:** La date et l'heure prévues de la commande, au format "YYYY-MM-DD HH:MM" (chaîne de caractères).
- **montant_payé:** Le total financier de la commande (nombre décimal).
- **remise:** Le montant de la réduction appliquée (nombre).
- **livreur:** L'identifiant de l'utilisateur (rôle livreur) assigné à cette course (nombre, ou null si aucun).
- **notation:** La note sur 5 laissée par le client (nombre, ou null si non noté).
- **commentaire:** Un éventuel avis écrit laissé par le client (chaîne de caractères, ou null si aucun).

[Phase 3]

Dans cette troisième phase de notre projet, notre équipe s'est concentrée sur le côté dynamique du front-end, avec une forte utilisation de JavaScript et la mise en place de requêtes asynchrones.

La première étape a été d'intégrer un changement de charte graphique accessible via un bouton sur l'interface, permettant de modifier l'apparence visuelle sans avoir à recharger la page. Le choix de l'utilisateur est mémorisé grâce à un cookie, garantissant que le thème sélectionné reste actif lors de la navigation ou qu'il revienne au mode par défaut si le cookie est absent ou incohérent.

Concernant l'interface et les formulaires, bien que nous ayons tenu à concevoir nos menus déroulants en pur CSS pour répondre à nos exigences d'intégration, nous avons implémenté les vérifications JavaScript demandées côté client pour les formulaires d'inscription et de connexion. Ainsi, les données ne sont envoyées au serveur que si l'ensemble des champs (email, format des données, etc.) est correctement rempli. Nous avons également enrichi l'expérience utilisateur en ajoutant des compteurs de caractères en temps réel pour les champs limités, ainsi qu'une option permettant de masquer ou d'afficher les mots de passe.

Nous avons également profité de cette étape pour affiner certains détails de notre code PHP existant, notamment en ajustant la logique de notre header pour qu'il s'adapte dynamiquement à chaque vue (par exemple, en évitant d'afficher le bouton ou le lien vers la carte lorsque l'utilisateur se trouve déjà sur la page de la carte).

L'utilisation de requêtes asynchrones a transformé la fluidité de KØLD. Sur la page de notre carte, nous avons mis en place des tris qui s'effectuent instantanément sur les données déjà chargées, ainsi que des filtres asynchrones permettant de trier les plats par catégories, contraintes alimentaires ou par prix.

Côté gestion, plusieurs nouvelles fonctionnalités ont été ajoutées sans nécessiter de rechargement de page :

- Les clients peuvent désormais modifier les informations de leur profil de manière asynchrone.
- Un client peut ajouter ou retirer des articles sur une commande déjà payée mais qui n'est pas encore en préparation ; le montant se met alors à jour automatiquement pour demander un paiement additionnel si nécessaire.
- Après la livraison, le client a la possibilité de laisser une note unique sur sa commande.
- Le restaurateur peut changer le statut des commandes (en préparation, prête) et les assigner directement aux livreurs disponibles.
- Le livreur, depuis son interface, peut valider la fin d'une livraison en temps réel.
- Enfin, l'administrateur dispose désormais du pouvoir de bloquer ou débloquer un utilisateur via des requêtes asynchrones. Si un compte est bloqué, sa session est immédiatement interrompue , bien que ses commandes déjà payées puissent tout de même poursuivre leur cycle de livraison.